

Be CIM or Be Memory: A Dual-mode-aware DNN Compiler for CIM Accelerators

Shixin Zhao

Institute of Computing Technology,
Chinese Academy of Sciences,
University of Chinese Academy of
Sciences
Beijing, China
zhaoshixin18@mails.ucas.ac.cn

Yuming Li

Institute of Computing Technology,
Chinese Academy of Sciences,
University of Chinese Academy of
Sciences
Beijing, China
liyuming22@mails.ucas.ac.cn

Bing Li

Institute of Microelectronics, Chinese
Academy of Sciences
Beijing, China
libing2024@ime.ac.cn

Yintao He

Institute of Computing Technology,
Chinese Academy of Sciences,
University of Chinese Academy of
Sciences
Beijing, China
heyintao19z@ict.ac.cn

Mengdi Wang

State Key Lab of Processors, Institute
of Computing Technology, Chinese
Academy of Sciences
Beijing, China
wangmengdi@ict.ac.cn

Yinhe Han

State Key Lab of Processors, Institute
of Computing Technology, Chinese
Academy of Sciences
Beijing, China
yinhes@ict.ac.cn

Ying Wang*

State Key Lab of Processors, Institute
of Computing Technology, Chinese
Academy of Sciences
Beijing, China
wangying2009@ict.ac.cn

Abstract

Computing-in-memory (CIM) architectures demonstrate superior performance over traditional architectures. To unleash the potential of CIM accelerators, many compilation methods have been proposed, focusing on application scheduling optimization specific to CIM. However, existing compilation methods often overlook CIM’s capability to switch dynamically between compute and memory modes, which is crucial for accommodating the diverse memory and computational needs of real-world deep neural network architectures, especially the emerging large language models. To fill this gap, we introduce CMSwitch, a novel compiler to optimize resource allocation for CIM accelerators with adaptive mode-switching capabilities, thereby enhancing the performance of DNN applications. Specifically, our approach integrates the compute-memory mode switch into the CIM compilation optimization space by introducing a new hardware abstraction

attribute. Then, we propose a novel compilation optimization pass that identifies the optimal network segment and the corresponding mode resource allocations using dynamic programming and mixed-integer programming. CMSwitch uses the tailored meta-operator to express the compilation result in a generalized manner. Evaluation results demonstrate that CMSwitch achieves an average speedup of 1.31× compared to existing SOTA CIM compilation works, highlighting CMSwitch’s effectiveness in fully exploiting the potential of CIM processors for a wide range of real-world DNN applications.

CCS Concepts: • Hardware → Memory and dense storage; • Software and its engineering → Compilers.

Keywords: Compute-in-memory (CIM), Compilation, Deep Neural Network (DNN)

ACM Reference Format:

Shixin Zhao, Yuming Li, Bing Li, Yintao He, Mengdi Wang, Yinhe Han, and Ying Wang. 2025. Be CIM or Be Memory: A Dual-mode-aware DNN Compiler for CIM Accelerators. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS ’25)*, March 30–April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3676641.3716248>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS ’25, March 30–April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1079-7/25/03.

<https://doi.org/10.1145/3676641.3716248>

1 Introduction

The Computing-In-Memory (CIM) architecture is highly regarded for enabling in-situ computation [3, 5, 8, 9, 15, 38, 41].

CIM minimizes frequent data transfers and enhances the parallel execution of matrix multiply-and-accumulate (MAC) operations, resulting in notable performance improvements. Compared to conventional architectures, CIM significantly mitigates persistent memory wall problem [49] and demonstrates strong competitiveness in data-intensive applications especially deep neural network (DNN) inference [38, 43, 52].

To enhance efficiency and fully realize the potential of the CIM accelerators, researchers have explored various compilation optimization techniques, aimed at various CIM architectures such as resistant RAM and SRAM-based solutions [14, 16, 21, 33, 39, 44]. These compilation tools significantly reduce entry barriers for users adopting the CIM architecture and support the widespread deployment of CIM chips. Earlier compilation tools for CIMs assume that neural network weights are pre-loaded into memory. These tools formulate an optimized policy for weight mapping and devise a computation scheduling scheme across various operational granularities to maximize memory resource utilization and enhance computational performance. [3, 33, 39]. Although there have been significant improvements in compilation optimization techniques for CIM architecture, current methods still consider the memory and compute resources on the chip static, which does not accurately represent the modern advancements in CIM designs. In practice, many modern CIM designs feature dual-mode memory arrays that can dynamically switch between memory and compute modes [19, 24, 51, 53]. As depicted in Figure 1(a), the CIM array transitions between these modes by resetting the input driver. This dynamic functionality broadens the compiler's optimization possibilities for CIM mapping, enhancing DNN application performance. Previous compiler-level optimization efforts did not fully exploit these opportunities, missing out on the benefits of dual-mode CIM arrays.

Moreover, different real-world DNN architectures have distinct memory and computation resource requirements. Figure 1 (b) depicts the varying demands on the memory and computation resource of different DNN models (*i.e.*, CNNs [22], LLaMA [45], GPT [6], *etc.*) to reach the optimal performance on the CIM chip. Convolutional neural networks (CNNs) have relatively high arithmetic intensity (FLOPs/Memory OP) and demand a higher ratio of compute to memory resources on CIM. For instance, ResNet50 has an average arithmetic intensity of 66, and its performance reaches the highest point when the ratio of compute to memory resource reaches almost 80%. Thus, some typical CNNs require more CIM arrays working in compute mode, when they already have sufficient CIM arrays configured as the on-chip scratchpad memory for activation caching. In contrast, Transformer-based models typically have much lower arithmetic intensity. For instance, the generative model LLaMA 2 has an average arithmetic intensity of around 2 for single batch inference and Figure 1 (b) depicts that LLaMA 2 garners the best performance when the ratio of compute to memory in CIM arrays

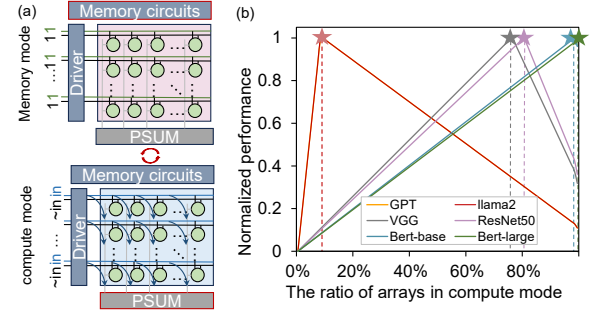


Figure 1. (a) CIM switching between memory and compute mode by setting up control signals to the input driver; (b) Normalized performance variation with the ratio of arrays in compute mode changes. Please note that putting more CIM arrays in compute mode deprives them of the chance of working as scratchpad memory for storing and loading intermediate data, *e.g.* activations. CIM arrays in compute mode must store static data, *i.e.* pre-determined weights.

is about 10%, which means it is better to offer more on-chip random memory for activations and KV cache rather than to increase compute-power, given that it is almost impossible to cache all the massive parameters of large language models on a single CIM chip. Although this conclusion drawn from Figure 1 only makes sense for certain models and hardware configurations, like the on-chip memory space, main memory bandwidth, *etc.*, it reveals a fact that it is not necessarily correct to assume all CIM arrays should be put at the compute mode as in prior compilation works. Moreover, the requirements for memory and compute CIM arrays of the same model may vary across different layers or stages of execution. Therefore, a compiler customized for dual-mode CIM that optimizes the memory and compute mode of the CIM array is significant.

In this work, we propose a novel CIM compiler that takes the CIM mode switch into account and co-adjusts the CIM working mode and the mapping of the DNN applications in the context of dual-mode CIMs. Specifically, for a given target CIM architecture and the neural network workload, the proposed compiler can determine the arrays' mode being the compute or memory, and the optimal allocation of those arrays. Once the mode-switch decision is made, the compiler also schedules operators on the respective arrays to achieve optimal performance.

However, to achieve this goal, we have to address the following two challenges: (1) **Exponential space expansion:** In dual-mode CIM, each CIM array can work in memory or compute mode. Consequently, the problems of array mode selection and weight mapping are entangled in the deployment of target DNNs, which constitutes a larger exploration space for the compiler. For instance, with m arrays in CIM,

there are 2^m choices of the mode allocation during the compilation. It is proved that model scheduling for CIM is already a complicated problem with the optimization space about polynomial complexity, dual-mode CIM will face a 2^m times larger space with an exponential level. Therefore, we have to formalize such a jointly optimized exploration space that combines the allocation and mapping decisions, along with a search strategy when designing our compiler. (2) **Dual-mode switching schedule:** When scheduling each DNN operator in a dual-mode CIM, we should deliberately determine the mode of arrays as the number of arrays in different modes also affects the efficiency of the current operator and the scheduling of subsequent operators. Thus, the compiler for dual-mode CIM must account for the interdependence between array mode scheduling and weight mapping. The separate treatment of mapping and scheduling in previous compilers for CIM with fixed-mode arrays is insufficient for enhancing performance in dual-mode CIM architectures. Thus, we propose a holistic optimization framework that integrates DNN mapping and array mode scheduling for dual-mode CIMs.

To address these challenges, our approach at first provides the hardware abstraction of dual-mode CIM accelerators based on CIM-MLC [33] so that the dimension of CIM reconfiguration can be fused into the original mapping/scheduling space of CIM as a formalized optimization space. Second, to make the joint optimization problem tractable for modern large-scale neural networks, we employ a divide-and-conquer two-step policy, co-optimizing the array mode switch, allocation, and mapping of neural networks. Given that CIM memory space often cannot accommodate the entire model on the chip, the network must be executed in segmented partitions in serial. This is a common trend with billion-scale large language models. We first utilize dynamic programming (DP) to network segmentation. The overhead introduced by the array mode switch is taken into account when applying DP for global optimization. Afterward, we use mixed integer programming to automatically explore the optimization space for operator mapping with tunable hardware resources within each segment. The compiled results are then output in a meta-operator flow marked with memory-compute switch information.

Specifically, the main contributions of this work include:

- To support various DNNs including nowadays large language models, we introduce CMSwitch, a novel dual-mode-aware CIM compiler that leverages the mode switch capability of compute/memory of CIM arrays to meet diverse DNN application requirements. We formalized the joint-optimization problem of mode-switch, mapping, and scheduling for standalone CIM accelerators, and released the first compiler aware of this important CIM feature.

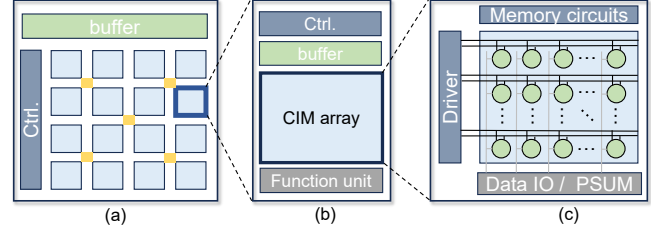


Figure 2. Hierarchical CIM architecture (a) CIM core with CIM arrays and the corresponding peripheral units; (b) CIM array and the corresponding peripheral units; (c) CIM array.

- We comprehensively consider the challenges and opportunities brought by mode switching. Without causing too much exploration overhead, we propose a two-step optimization strategy to make the compilation process converge at the optimal design point of the large joint design space. By formalizing the overhead and performance improvement introduced by CIM mode switch, we employ DP and MIP to determine the optimal network segment in temporal and allocate compute-memory resources in spatial.
- We evaluate the CMSwitch across a set of DNN benchmarks. Compared with state-of-the-art compilation works [33], CMSwitch achieves average inference speed improvement by 1.31×. We also verify CMSwitch for various workload scales, demonstrating robust dual-mode-aware compilation support for diverse DNN architecture demands. It is proved the proposed compiler shows especially great potential for popular large models that cannot be fitted into the on-chip memory.

2 Background

2.1 Computing-In-Memory Accelerator Architecture

As depicted in Figure 2 (a)-(c), independent CIM-based DNN accelerators are typically structured as hierarchical architecture comprising multiple CIM cores. Each core integrates a CIM array along with its peripheral buffer and circuitry. This design enables in-situ computation within the memory, thereby mitigating the data transfer bottlenecks commonly seen in conventional architectures that separate computation and memory. Prior researches [1, 3, 5, 8, 9, 15, 23, 25–27, 32, 37, 38, 41, 47] have proposed various CIM accelerators, providing robust support for high-performance computing and naturally aligning with large-scale parallel computing applications such as DNN inference.

2.1.1 Dual Modes CIM Array. The dual-mode CIM array can operate as both a memory and compute unit when applying a slight enhancement on the input or output drivers [2, 10, 18, 24, 42, 48, 51, 53].

As illustrated in Figure 3, switching between memory and compute modes of CIM arrays can be achieved by altering the

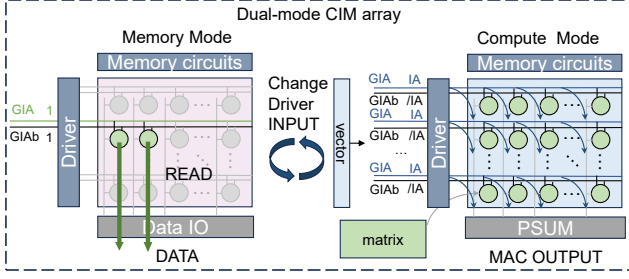


Figure 3. Dual-mode CIM Array.

array’s driver inputs, as demonstrated by DynaPlasia [24]. This mode-switching functionality is controlled by modifying the input signals on the global lines. When the Global Input Activation line (GIA) and its complement (GIAb) are set to a high state (1), the array functions in memory mode, allowing standard memory read-write operations. Conversely, when the GIA and GIAb are configured as input activation (IA) and inverse of input activation $\neg IA$, respectively, the array operates in compute mode, performing bit-series multiplication-addition operations.

2.1.2 CIM Compute Paradigm. When CIM arrays perform computations, they enable the multiply-accumulate (MAC) operations to be executed entirely within the array in parallel, as illustrated in the Figure 3 (right). This architecture inherently supports matrix-vector multiplication (MVM) and matrix-matrix multiplication (MMM). In the case of MVM, the matrix is mapped onto the CIM array, while the vector serves as the array input. The multiplication is performed within each cell, with accumulation occurring along the bit-lines or at the output side, producing MVM results directly from the array. Many classic DNN operators, such as fully connected layers and convolutions, can be readily transformed into MMM or MVM operations. For instance, while convolutional kernels cannot be directly mapped onto the array, the convolution operations can be unrolled into an equivalent matrix-matrix multiplication (MMM). This equivalent MMM is subsequently mapped and executed on the CIM array, following the standard MMM procedure.

2.2 CIM Compilation Works for DNN

With the increasing attention on CIM, there has been a significant surge in efforts to develop a compilation optimization stack aimed at facilitating the deployment of DNN algorithms across various CIM architectures [3, 33, 39, 44].

Existing compilation optimization approaches for CIM predominantly emphasize scheduling optimizations, such as task mapping, resource allocation, and dataflow scheduling, to fully exploit the static on-chip resources of CIM chips, thereby reducing latency. For example, OCC [39], built upon MLIR, utilizes a specific ISA to support scheduling optimization for multiple operators. CIM-MLC [33] addresses the

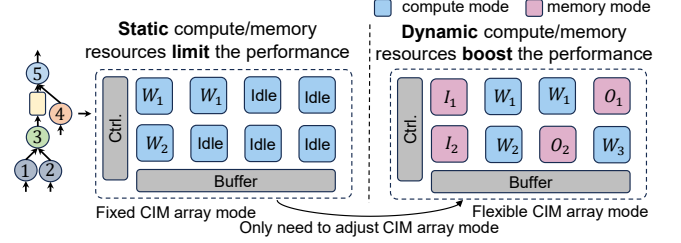


Figure 4. (a) Existing typical CIM mapping method that treats all the CIM arrays as compute arrays; (b) Dual-mode-aware mapping method.

challenges posed by multi-level and heterogeneous program interfaces in CIM accelerators, implementing weight duplication and pipelining techniques tailored for various CIM computation modes. These compilation strategies alleviate the programming complexity of utilizing CIM processors, accelerate application deployment, and allow researchers to focus more on architecture design.

However, existing compilers overlook a crucial aspect: the dual-mode capability of CIM arrays, resulting in suboptimal performance. As shown in Figure 4, when taking the dual-mode feature of CIM arrays into account, the compiler can dynamically allocate compute and memory resources (b). Thus, it can enhance the DNN performance when keeping more data on the chip by switching the CIM arrays to memory mode. Instead, the traditional compiler has to move these data to off-chip memory, incurring extra latency.

In summary, the dual-mode capability of CIM arrays introduces a powerful mechanism for dynamically adjusting on-chip resources to meet the diverse computation and memory demands of various DNN workloads. By intelligently switching CIM arrays between compute and memory modes, we can flexibly allocate resources based on the specific requirements of different DNN inference tasks, ultimately optimizing performance.

3 Motivation

This section describes the motivation behind developing a dual-mode-aware DNN compiler for CIM accelerators. We identify the diverse on-chip computing and memory requirements inherent in real-world DNNs. Additionally, we discuss the opportunities of meeting these application requirements through the optimization of CIM array mode configuration during compilation.

3.1 Insights into Diverse DNN Requirements

Variations among different network architectures. Mainstream neural networks exhibit diverse architecture designs, leading to varied hardware requirements [12, 13, 22, 29, 34, 35, 40, 45]. Figure 5 (a)(b) illustrates the normalized performance variation heatmap of Llama2 [45] and ResNet-50 [22] with changes in the number of arrays in compute/memory

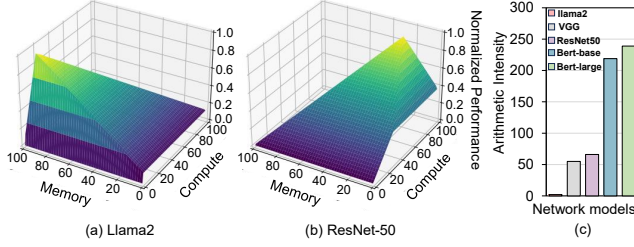


Figure 5. (a)(b) Normalized performance variation with the changes of compute/memory array; (c) Arithmetic Intensity.

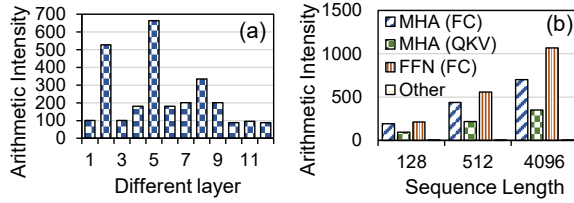


Figure 6. (a) Layer-specific arithmetic intensity of ResNet-50 [22]; (b) Arithmetic intensity of BERT-Large with different sequence lengths [12].

mode. We assume there is a total of 100 dual-mode CIM arrays on-chip, where the switchable CIM array is as Dynapalsis [24]. The memory and compute axes represent the number of CIM arrays in memory and compute mode, respectively. The vertical axis indicates the theoretical performance normalized to the optimal performance with the same amount of total arrays. Green indicates better performance, while dark blue indicates poor performance. Llama2 and ResNet-50 exhibit distinct preferences for hardware resource allocation, stemming from their differing arithmetic intensities. As illustrated in Figure 5(c), ResNet-50 has a significantly higher average arithmetic intensity compared to Llama2. Consequently, Llama2, with its lower arithmetic intensity, does not require extensive computing resources but necessitates increased on-chip memory to complete its operations. Thus, Llama2 demands more CIM arrays in memory mode to meet its computational needs. Conversely, ResNet-50, with its higher arithmetic intensity, benefits from greater compute resources to achieve optimal performance. Furthermore, Figure 5(c) indicates the varying arithmetic intensities across different models. Therefore, the dynamic adjustment of memory and compute resources in CIM is essential to provide optimal performance for diverse DNNs.

Layer-wise variations within the same network. Within the same neural network, different layers also have varying hardware demands due to factors such as input data size and network parameters, including weight kernel size. For example, as illustrated in the Figure 6(a), ResNet-50[22] comprises four distinct blocks, each containing three configurations of convolution layers. The arithmetic intensity of

these three layers varies significantly, ranging from below 100 FLOPs/MOP to over 700 FLOPs/MOP.

Variations on different workload scales. Transformer-based [46] NLP models, such as BERT [12], show dynamic resource requirements based on varying input and output sequence lengths. As illustrated in the Figure 6(b), the arithmetic intensity of the model fluctuates significantly with the input and output sequence length, varying from under 150 FLOPs/MOP to over 1000 FLOPs/MOP. Additionally, different computation stages within the models, such as fully connected (FC) layers and query-key-value (QKV) computations, display varying arithmetic intensities. For example, FC layers demonstrate much higher arithmetic intensity compared to QKV computations as the sequence length increases. As sequence length grows, more memory is needed to store intermediate states and longer contextual information, while additional computational resources are required to manage the increased complexity of the attention mechanism. Consequently, the demand for compute and memory resources dynamically adjusts with sequence length.

3.2 Opportunity of CIM Dual-Mode Switch

Given the diverse resource demands of various DNN architectures, layers, and workload scales, dynamic hardware resource allocation is crucial for optimizing model execution performance. A static compute/memory resource ratio is often insufficient to achieve optimal efficiency across different scenarios. Traditional compilation techniques typically struggle with inefficiencies due to their inability to adapt to fluctuating resource requirements. By leveraging the dual-mode switching capability of CIM arrays, we can dynamically alternate between compute and memory modes, enabling CIM accelerators to more effectively accommodate the diverse needs of DNN workloads. Specifically, repurposing compute arrays into memory arrays allows CIM accelerators to expand on-chip memory resources, which is particularly beneficial for storing dynamically generated activations in DNNs. This flexibility can significantly boost overall system performance and energy efficiency by tailoring hardware configurations to the unique requirements of each DNN model.

To leverage this flexibility, we propose CMSwitch, a dual-mode-aware DNN compiler for CIM processors, ensuring that the dual-mode CIM arrays provide optimal performance for any given workload. In the following sections, we will introduce the workflow of CMSwitch, elaborating on the dual-mode-aware compilation optimization pass.

4 Dual-Mode-Oriented Compilation Stack

4.1 Overall Workflow

Figure 7 illustrates the workflow of CMSwitch, which takes user-defined hardware parameters and neural network applications as inputs. The neural network is initially converted

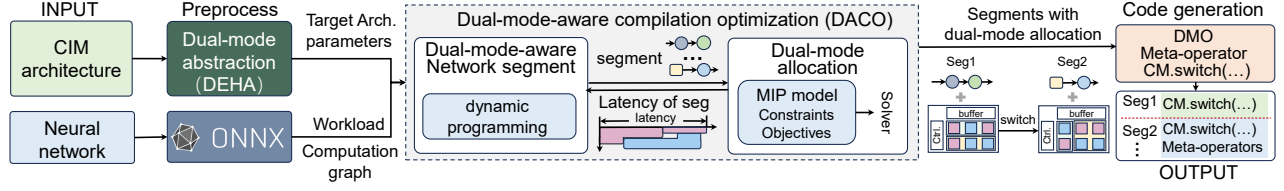


Figure 7. Overview of dual-mode-aware compilation process.

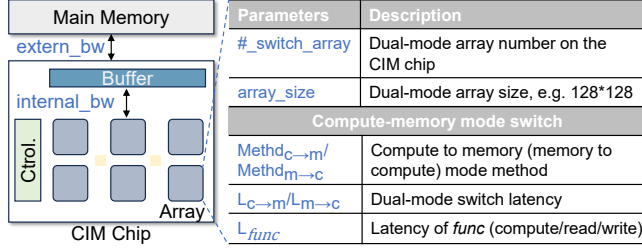


Figure 8. Dual-mode Enhanced Hardware Abstraction.

into ONNX format [4], lowering it to a computation graph expression. To integrate compute-memory mode switching into the compilation optimization space, we incorporate the dual-mode functionality of arrays in the hardware abstraction. This is achieved by introducing the methods and overheads associated with compute-memory mode switching into the hardware abstraction parameters.

During compilation optimization, to minimize application latency within the joint optimization space, CMSwitch develops a divide-and-conquer two-step policy. CMSwitch first decides network segmentation that accounts for dual-mode switch overheads, and then optimizes the dual-mode resource allocation and scheduling for operators within each segment. Through iterative exploration and optimization using Dynamic Programming (DP) and Mixed-Integer Programming (MIP), CMSwitch derives the globally optimal network segmentation schedule, along with the corresponding resource allocation and mapping results for each operator.

Furthermore, to effectively present our compilation results, we introduce meta-operators specifically designed for dual-mode switching. These meta-operators facilitate the output of compilation results that incorporate the compute-memory switch scheme. Upon obtaining the memory-compute mode switch plan offline, the actual dual-mode switch needs to be executed online with the support of the dual-mode CIM.

4.2 Dual-Mode Enhanced Hardware Abstraction (DEHA)

Hardware abstraction is crucial to providing essential hardware information to the compiler for the compilation process. As shown in the Figure 8, we incorporate the dual-mode parameter of the array and the related switch function into the hardware abstraction. Together with architecture parameters, this abstraction enables CMSwitch to access the

optimization space of the dual-mode CIM array and relevant architecture parameters.

Dual-mode CIM architecture. In abstracting the CIM architecture, we model the CIM chip hierarchically. Given our focus on optimizing the dual-mode CIM, we simplify the abstraction to include only two essential tiers: chip and array. At the finest granularity, our abstraction is at the CIM array level, which represents the smallest hardware unit capable of mode switching. Users are required to define parameters such as the number of dual-mode arrays, the array sizes, internal bandwidth, and external global bandwidth, all of which significantly influence the behavior and performance of the CIM chip.

Dual mode switch. During the compilation optimization process, it is crucial to consider both the functionality and overhead of the dual-mode switch to enable the compiler to balance the benefits of mode switching and explore optimal performance results. To facilitate the compilation process, it is essential to define the method and overhead of the compute-memory mode switch adopted by the target chip at the hardware abstraction stage. This may involve techniques such as altering the inputs of wordlines or bitlines. Additionally, the overhead of the compute-memory mode switch is assessed at the granularity of the switchable arrays. These parameters significantly impact subsequent compilation optimization solutions.

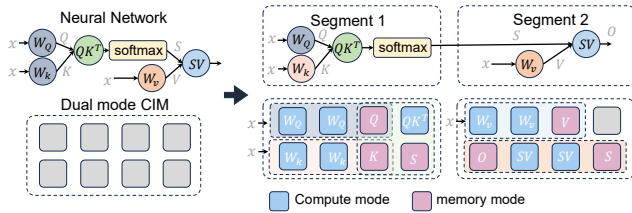
When CIM arrays operate in different modes, executing a single operation—such as activating an array for computation or data reading—may incur varying overheads. To account for this, we introduce an option in the hardware abstraction parameters to record the overhead of computation and memory operations. This approach ensures that the associated overheads of read-write and computation of arrays are considered during the compilation optimization process, thereby evaluating the impact of different modes on overall performance.

4.3 Dual-Mode-Aware Compilation Optimization

Based on the abstraction of the hardware architecture, we proposed the Dual-Mode Aware Compilation Optimization (DACO) to allocate the dual-mode CIM arrays for each CIM-supported operator, to minimize overall execution latency. This phase gets the ONNX-format DNN and hardware abstraction as input. We employ a divide-and-conquer two-step

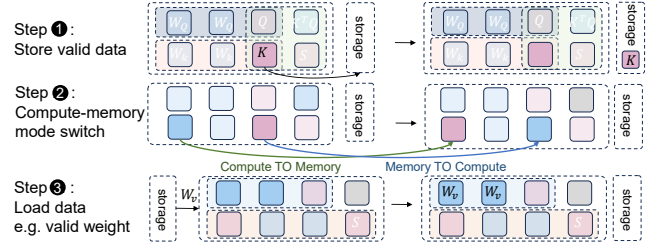
Table 1. Notations Used in Dual-mode Compilation.

Variables - decide during optimization	
$\lambda_{min}(i, x, y)$	taking value 1 if CIM array (x,y) is assigned to O_i in memory mode as input buffer, 0 otherwise.
$\lambda_{mout}(i, x, y)$	taking value 1 if CIM array (x,y) is assigned to O_i in memory mode as output buffer, 0 otherwise.
$\lambda_c(i, x, y)$	taking value 1 if CIM array (x,y) is assigned to O_i in compute mode, 0 otherwise.
Mem_{O_i}	the number of CIM array in memory mode that O_i has. $Mem_{O_i} = \sum_{x,y} \lambda_{min}(i, x, y) + \sum_{x,y} \lambda_{mout}(i, x, y)$
Com_{O_i}	the number of CIM array in compute mode that O_i has. $Com_{O_i} = \sum_{x,y} \lambda_c(i, x, y)$
$Switch_{m \rightarrow c}$	arrays from memory mode to compute mode for segment S' to S , $Switch_{m \rightarrow c} = \sum_{x,y} (\sum_{O_i \in S'} \lambda_m(i, x, y) \odot \sum_{O_j \in S} \lambda_c(j, x, y))$, where $\lambda_m(i, x, y) = \lambda_{min}(i, x, y) + \lambda_{mout}(i, x, y)$
$Switch_{c \rightarrow m}$	arrays from compute mode to memory mode for segment S' to S , $Switch_{c \rightarrow m} = \sum_{x,y} (\sum_{O_i \in S'} \lambda_c(i, x, y) \odot \sum_{O_j \in S} \lambda_m(j, x, y))$
Constants - determine by application and CIM chip initially	
IN_{O_i}/OUT_{O_i}	input data/output data of operator O_i
AO_i	arithmetic intensity of operator O_i
N_{cim}	the number of dual-mode switchable CIM array
OP_{cim}	operation/cycle a CIM array can provide, $OP_{cim} \propto array_size$
D_{cim}	data/cycle a CIM array can provide in the memory mode, which is impacted by architecture design and user-defined topology.
D_{main}	data/cycle main memory and original on-chip buffer can provide, $D_{main} \propto extern_bw + internal_bw$

**Figure 9.** Illustration of Network Segment.

policy. First, dynamic programming (DP) is utilized to partition the network into multiple segments, taking into account the switching overhead between segments. Following segmentation, mixed linear programming (MIP) is applied to co-optimize the allocation of on-chip computing/memory resources and scheduling for operators, adhering to the constraint of total available resources. During this optimization process, the dual-mode CIM arrays are dynamically adjusted to either memory or compute mode based on the optimization objective. Finally, this phase outputs the network segmentation results along with the corresponding allocation of dual-mode CIM array in memory and compute mode for each segment. The notation we used for optimization space formalization in this section is summarized in Table 1.

4.3.1 Dual-Mode-Aware Network Segment. In the network segment step, we organize the network compute operators at the granularity of network segments, being allocated on the dual-mode CIM arrays holistically. Network segmentation offers two key benefits: 1) it reduces the optimization space from an exponential size, considering the entire number of operators, to a more feasible one, and 2) it facilitates the accommodation of real-world DNN networks that cannot

**Figure 10.** Three sources of inter-segment overhead.

fit entirely on the chip to be optimized on the dual-mode CIM. As shown in the Figure 9, taking the attention layer as an example, if we segment the attention process into two segmentations, we will execute them serially. The first segment maps to the hardware and executes, followed by the second segment after the first completes. Both of them will have their resource allocation plan. To further refine the segmentation search space, we employ a dynamic programming approach to optimize network latency in the context of scheduling network segments.

For a network N with m CIM-supportable operators (e.g., MVM and MMM), we first topologically sort these operators, denoted as $O_1, O_2, \dots, O_m$, where O_i and O_j ($i, j \in \{1, 2, \dots, m\}$) satisfy that if $i < j$, then O_i is completed no later than O_j . The dependency relationship between operators is denoted as W , where $w_{i,j} \in W$ indicates that the output of O_i is input to O_j . The segments after segmentation are denoted as $S_{i,j}$, indicating that operators from O_i to O_j belong to the same segment $S_{i,j}$. For operators that cannot fit directly onto the CIM accelerator, we will partition them into smaller sub-operators. This partitioning process uses a greedy strategy, with the partition granularity determined by the available on-chip resources. This approach ensures that each resulting sub-operator can be fully mapped onto the chip. Finally, we replace the original operator in the sorted operator list with these sub-operators, enabling efficient execution on the CIM accelerator. To minimize inference latency, the segmented network execution overhead includes both intra-segment execution latency and inter-segment mode-switch latency.

Intra-segment latency. Within each segment, all operators are mapped to the CIM chip simultaneously. Therefore, optimization methods such as pipelining can be employed to minimize the overall latency within the segment. The introduction of dual-mode CIM arrays makes the intra-segment latency highly dependent on on-chip memory and computation resource allocation. To optimize the intra-segment overhead, we will detail the dual-mode-aware resource allocation algorithm in the following subsection. We denote the intra-segment latency with resource allocation plan A as $T_{i,j}^{intra}(A)$ for $S_{i,j}$.

Inter-segment latency. Inter-segment latency encompasses the latency caused by switching CIM memory and computation modes and on-chip and off-chip data swapping. Specifically, as illustrated in Figure 10, the inter-segment mode switch mainly consists of three steps, storing the valid on-chip data to storage, performing the mode switch between the compute/memory, and loading data. We formalize the inter-segment latency based on the two adjacent segments, $S_{k,i-1}$ with the resource allocation plan A' and $S_{i,j}$ with the resource allocation plan A ($k < i \leq j$).

For step one: if segment $S_{k,i-1}$ contains more memory arrays than $S_{i,j}$, and the output data from the segment $S_{k,i-1}$ is needed for subsequent computations, this data must be written back to the main memory before switching this CIM arrays from memory mode to compute mode. We denote the data store latency as $T_{i-1,i}^{wb}(A', A)$, and it can be estimated according to the data transfer volume and the memory bandwidth. For data that can be processed in place and will not be reused, such as softmax results in attention, the corresponding CIM arrays can be directly switched from memory to compute mode without data write-back.

For the second step, when switching between memory and compute modes, if the required arrays differ between adjacent network segments, the dual-mode switch overhead must be considered. When the $S_{k,i-1}$ uses arrays in compute mode while the $S_{i,j}$ requires them in memory mode, the overhead of switching those arrays from compute mode to memory mode must be accounted for, and vice versa. The overhead $T_{i-1,i}^{swc}(A', A)$ is detailed in the Eq. 1

$$T_{i-1,i}^{swc}(A', A) = L_{M \rightarrow C} \times \text{Switch}_{m \rightarrow c} + L_{C \rightarrow M} \times \text{Switch}_{c \rightarrow m}. \quad (1)$$

For the third step, as different segments process different operators, the weights stored in the compute arrays must be reloaded and rewritten. Each operator has unique weights, necessitating an update of the compute arrays' stored weights accordingly. The overhead $T_{i-1,i}^{rw}(A', A)$ is shown in the Eq. 2:

$$T_{i-1,i}^{rw}(A', A) = \max_{O_i \in S_{i,j}} \text{Com}_{O_i} \times \text{Latency_write}. \quad (2)$$

Based on the intra- and inter-segment overhead, network segmentation can be formalized as the following dynamic programming problem. As shown in Eq. 3, we denote the best network segment solution with the corresponding resource allocation plan A^* as the $L[m][A^*]$. The total cost of the network from operator 1 to m is the sum of the costs of two segments: from 1 to $i - 1$ and from i to m , plus the mode-switch latency between these segments. To reduce the solution space, any network segment exceeding the total on-chip resources is deemed invalid. If a segment requires more compute and memory arrays than the available on-chip resources, it must be further segmented during execution.

$$L[m][A^*] = \min_{1 \leq i < m} \{L[i][A'] + T_{i,m}^{intra}(A) + T_{i-1,i}^{inter}(A', A)\}, \quad (3)$$

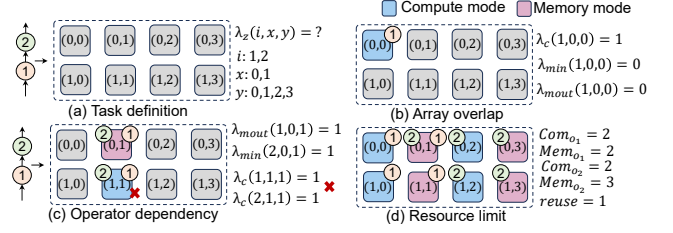


Figure 11. Constraints illustration for resource allocation.

where $T_{i-1,i}^{inter}(A', A)$ equals to

$$T_{i-1,i}^{inter}(A', A) = T_{i-1,i}^{wb}(A', A) + T_{i-1,i}^{swc}(A', A) + T_{i-1,i}^{rw}(A', A). \quad (4)$$

By traversing all potential choices and backtracking the segmentation plan according to Eq 3, we can efficiently find the optimal segmentation strategy, thereby minimizing execution time. This dynamic programming approach significantly reduces the search space complexity compared to exhaustive search methods.

4.3.2 Unified Dual-Mode Allocation with Scheduling.

In this section, we formalize the resource allocation and operator scheduling co-optimization problem as a Mixed-Integer Programming (MIP) problem to minimize the execution latency. For each network segment, the allocation optimization aims to determine the optimal modes of CIM arrays assigned to each operator. Operators within the segment are scheduled in the pipelined fashion to maximize computing parallelism. To capture the interdependence of compute and memory allocation and the scheduling for operators within a segment, we define specific objectives and constraints, where constraints dictate dual-mode CIM resources available for operators and objectives that optimize latency considering the pipeline structure, respectively. The details of our formulation are explained in the following contents.

Here we use the 2-d coordinate (x, y) to indicate the CIM arrays in the chip and use the $\lambda_z(i, x, y)$, $z \in \{\min, mout, c\}$ to indicate the resources allocation and mapping for operator O_i , where *min*/*mout* means array in memory mode as input/output buffer, respectively, and *c* means array in compute mode. When the CIM array (x, y) assigned to O_i is in memory mode as input and output buffer, $\lambda_{\min}(i, x, y) = 1$ and $\lambda_{mout}(i, x, y) = 1$, otherwise they are 0. Similarly, when the CIM array (x, y) assigned to O_i is in compute mode, $\lambda_c(i, x, y) = 1$. Our goal is to find the optimal $\lambda_z(i, x, y)$ for each segment under the constraints imposed by the resource allocation and the objective related to the pipeline scheduling strategy.

Constraints. As the dual-mode resource space is defined, some optimization rules should be considered to include only the valid CIM array allocation solutions. Here, we use S_* to indicate any possible network segment.

1) *Array overlap.* A CIM array can be either memory or compute mode for an operator, but not both. Therefore, for

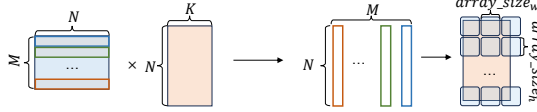


Figure 12. Example of MMM.

each CIM array (x, y) , if it is allocated to operator O_i , only one of $\lambda_{min}(i, x, y)$, $\lambda_{mout}(i, x, y)$, or $\lambda_c(i, x, y)$ can be 1.

$$\forall O_i \in S_*, \forall x, y : \lambda_{min}(i, x, y) + \lambda_{mout}(i, x, y) + \lambda_c(i, x, y) \leq 1. \quad (5)$$

As shown in Figure 11(b), CIM array $(0, 0)$ is assigned to O_1 in compute mode, so it can not be a memory array.

2) *Operator dependency.* If one operator's output O_i serves as the input of the next O_j , the output memory arrays of O_i can directly serve as the input memory arrays of O_j . Thus, a portion of the output memory buffer for O_i can be reused as an input memory buffer for O_j .

$$\forall O_i, O_j \in S_*, \text{if } w_{i,j} \in W :$$

$$\sum_{x,y} \lambda_{mout}(i, x, y) \cdot \lambda_{min}(j, x, y) < \frac{OUT_{O_i} \cap IN_{O_j}}{array_size}. \quad (6)$$

As the example shown in Figure 11(c), O_1 's output is the input of O_2 , where array $(0, 1)$ assigned to O_1 as output buffer can be input buffer of O_2 .

Otherwise, there are no reused CIM resources between the adjacent operators.

$$\forall O_i, O_j \in S_*, w_{i,j} \notin W, z \in \{mout, min, c\} :$$

$$\sum_{x,y} \lambda_z(i, x, y) \cdot \lambda_z(j, x, y) = 0. \quad (7)$$

As shown in Figure 11(c), CIM array $(1, 1)$ can not be compute array for operator O_1 and O_2 at the same time.

3) *Resource limit.* The total number of CIM arrays assigned to all operators must not exceed the available resources. $H_{i,j}(x, y)$ denotes $\lambda_{mout}(i, x, y) \cdot \lambda_{min}(j, x, y)$.

$$\sum_{O_i \in S_*} (Mem_{O_i} + Com_{O_i}) - \sum_{i,j} \sum_{x,y} H_{i,j}(x, y) < N_{cim}. \quad (8)$$

As shown in Figure 11(d), the allocated arrays are under the resource limit.

Objective function. Our objective is to optimize the allocation of arrays in memory or compute mode to minimize the execution latency of the network segment under a pipeline scheduling strategy. Through the pipeline, operators can execute in parallel. Thus, the latency of the segment can be approximated as the maximum execution time of any single operator within that segment. We denote the latency of O_i as L_{O_i} and get the following objective function:

$$\min \max L_{O_i}, \forall O_i \in S_*. \quad (9)$$

The system performance cost model for L_{O_i} estimates off-chip data access latency and on-chip computation latency based on the allocated compute and memory arrays. To quickly estimate the latency L_{O_i} , we develop a latency model as shown in Eq. 10. It is a function of the number of allocated compute and memory arrays (Com_{O_i} and Mem_{O_i}).

When we allocate Com_{O_i} compute arrays for O_i , they support $C = Com_{O_i} \cdot OP_{cim}$ computation amount per cycle, where OP_{cim} denotes the computation amount per cycle a CIM array can provide. When we allocate Mem_{O_i} memory arrays, they can access $Mem_{O_i} \cdot D_{cim}$ data per cycle, where D_{cim} represents the data per cycle a CIM array can provide. Combined with the data from storage and the original buffer (D_{main}), the accessible data per cycle is $Mem_{O_i} \cdot D_{cim} + D_{main}$. Given the arithmetic intensity of O_i (AI_{O_i}), this supports $M = (Mem_{O_i} \cdot D_{cim} + D_{main}) \cdot AI_{O_i}$ computation amount. The smaller value between the C and M determines the effective computation amount per cycle. L_{O_i} can be estimated by the total computation amount (OP_{O_i}) and the $\min(C, M)$. Therefore, when AI_{O_i} is large, it is preferable to allocate more compute arrays. Conversely, when AI_{O_i} is small, it is better to allocate more memory arrays.

$$L_{O_i} \propto \frac{OP_{O_i}}{\min(Com_{O_i} \cdot OP_{cim}, (Mem_{O_i} \cdot D_{cim} + D_{main}) \cdot AI_{O_i})}. \quad (10)$$

Taking matrix-matrix multiplication(MMM) as an example, as shown in Figure 12, for an operator O_i , the computation amount a CIM array can provide is: $\frac{N \times K}{\lceil \frac{N}{array_size_h} \rceil \times \lceil \frac{K}{array_size_w} \rceil}$. Meanwhile, N data can support $N \times K$ MAC computations, which means $AI_{O_i} = K$. The total number of multiply - accumulate operations needed is $OP_{O_i} = M \times N \times K$.

Using the optimization solver Gurobi [20], we solve the most efficient memory/compute mode allocation for each network segment, thereby optimizing the overall performance of the dual-mode CIM processor.

Algorithm 1 summarizes the dual-mode-aware compilation optimization (DAMO) workflow, detailing the interaction between network segment and resource allocation. Once the network segment and CIM array allocation plans are established, we perform post-allocation optimization, such as weight duplication, commonly used in CIM compilation optimization [33], to further enhance kernel mapping.

4.4 Dual-mode Support in Meta-Operator (DMO)

Once the frontend optimization outputs the network segmentation and the allocation strategy of memory/compute resources, CMSwitch uses meta-operator flow to express the compilation result. To accommodate the diverse methods for performing these switches, we use a meta-operator flow rather than machine code for better generality. Additionally, it can be integrated into other backends [33]. The introduced meta-operators and their corresponding syntax are shown in Figure 13. We add the *CM.switch* operator, which supports two types, *TOM* and *TOC*, representing mode switching at the granularity of CIM arrays. When the meta-operator type is *TOM/TOC*, it means switching the *array_addr* to memory/compute mode. When a CIM array is converted to memory mode, it is marked as a valid memory unit, effectively serving as an on-chip buffer. Alongside the new dual-mode

Algorithm 1 Summary of DACO.

```

1: Input: Neural network  $N$  in ONNX format and CIM hardware abstraction.
2: Output: The network segment  $S$  and the corresponding dual-mode CIM array allocation  $A^*$  for each segment.
3: // Preprocess graph.
4:  $(O_1, \dots, O_m) \leftarrow \text{Flatten}(G)$ 
5: // Run the network segment dynamic programming
6:  $L[0][\cdot] \leftarrow 0$ 
7: for  $1 \leq i \leq j \leq m$  do
8:   //Impossible cases are skipped to reduce search space
9:   if  $\min \#$  of CIM array  $S_{i,j}$  required  $< N_{cim}$  then
10:    // Run the resources allocation.
11:     $T_{i,j}^{intra}(A) \leftarrow MIP(S_{i,j})$  according to Eq 9.
12:   else
13:     $T_{i,j}^{intra}(\cdot) \leftarrow \infty$ 
14:   end if
15:   Compute  $T_{i-1,i}^{inter}(A', A)$  according to Eq 4.
16:    $L[j][A] \leftarrow \min(L[i][A'] + T_{i,j}^{intra}(A) + T_{i-1,i}^{inter}(A', A), L[j][A])$ 
17:    $S_{record}.update(i,j)$ 
18: end for
19:  $L_{min} = \min L[m][A^*]$ 
20: return backtrack( $L, S_{record}, L_{min}$ ) for network segment plan  $S$  and corresponding  $A^*$ .

```

```

<code> ::= <operators>* | parallel { " <operators>* " } "
<operators> ::= <operators>* <CIM>* <MEMORY>* <SWC>*
<SWC> ::= CM.switch (<type>, array_addr)
<type> ::= TOM | TOC

```

Figure 13. Syntax of dual-mode enhanced code generation.

switch meta-operators, we use standard operators to represent normal computation and memory access. Additionally, we use *parallel*{ } to represent the network segment as operators in a segment will execute in parallel. Users can convert the meta-operator flow into ISA or machine code suitable for their specific CIM chips.

5 Evaluation

5.1 Setup

Simulator. For the functional simulation, we adopt the CIM-MLC functional simulator [33]. We use the functional simulator to execute the generated meta-operator flows within the CIM architecture. By comparing the execution result with the PyTorch framework [31], we verify the effectiveness of our compilation results. To evaluate execution latency, we built our simulator upon existing open-source simulators [7, 50], incorporating necessary modifications to simulate the dual-mode switch. We also modified the hardware configuration to align with the specifications of Dynaplasia [24], our target hardware.

Table 2. CIM Architecture Configuration.

Parameter	Configuration
#_switch_array	96
array_size	320 × 320
buffer_size	10KB×8
internal_bw	32b/cycle
Method _{c→m} / Method _{c→m}	change the input of global IA and IA'
L _{c→m} / L _{c→m}	1 cycle

CIM architecture configuration. Our target CIM architecture configuration is based on Dynaplasia [24], a real CIM chip that supports the compute-memory mode switch. The main parameters of the architecture are listed in the Table 2. The CIM mode switch of Dynaplasia is achieved by altering the global wordline input. Consequently, the actual execution of the *CM.switch* operator at the runtime is setting the corresponding signal to the global wordline.

Network benchmark. To verify both the efficiency and generality of CMSwitch, we use various types of neural networks as our network benchmark. For convolution-based architecture, we use the classic MobileNet [36], ResNet [22], and VGG [40] series, tested on the ImageNet dataset[11]. For the Transformer-based architecture, we use the encode-only model BERT [12] and decode-only models OPT [54] and LLaMA 2 [45]. All models are quantized with 8-bit precision for weights and activations.

Baseline. To evaluate the benefits of dual-mode switching for application execution, we compare the compilation results of CMSwitch with existing CIM compilation works. We adopt three compilers with different optimization strategies as baselines to demonstrate the effectiveness: PUMA [3] (focusing on operator duplication and pipeline scheduling), OCC [39] (optimizing operator mapping via tiling and loop unrolling), and CIM-MLC [33] (employing multi-grained pipelining and operator duplication for diverse architecture). Our primary comparison metric is the execution latency of the compiled applications.

5.2 End-to-End Performance

We first evaluated the end-to-end performance of CMSwitch, with the results presented in Figure 14. For transformer-based models, we set the sequence length to 64. Compared to the main baseline, state-of-the-art compilation work CIM-MLC [33], CMSwitch demonstrated performance improvements ranging from 1.02× to 1.25× (average 1.17×) for the encode-only BERT-large model. For the decode-only LLaMA2-7B model, CMSwitch yielded a performance gain of 1.13× to 1.30× (average 1.24×). For OPT-13B, CMSwitch outperformed the baseline by 1.20× to 2.03× (average 1.73×). For CNN models, CMSwitch delivered a performance boost of 1.06× to 1.23× for MobileNet, 1.07× to 1.23× for ResNet18, and 1.32× to 1.48× for VGG-16. Overall, CMSwitch achieved an average speedup of 1.31×, with a maximum improvement of 2.03× compared to CIM-MLC.

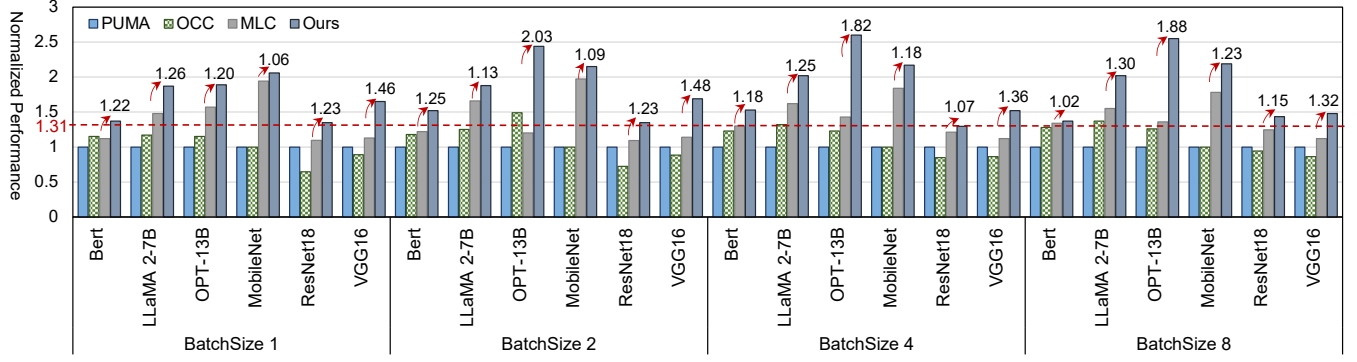


Figure 14. Speedup compared to the baselines. The red arrows with numbers highlight the speedup of CMSwitch compared to the main baseline, CIM-MLC[33]. The red line highlights the Geomean bar of performance improvement over CIM-MLC.

The fundamental enhancement brought by CMSwitch lies in its ability to expand the compilation optimization space for CIM accelerators. By leveraging the dual-mode switchable capabilities of CIM, CMSwitch dynamically adjusts the balance between computation and memory resources based on the application’s requirements, offering more flexible optimization options than all the baselines. This adaptability is particularly advantageous for DNN applications with substantial memory demands and varying arithmetic intensity.

Specifically, for large transformer-based benchmarks, where the hardware may be unable to fully map the weights, CMSwitch optimizes network segmentation by accounting for the overhead associated with switching between compute and memory modes. Meanwhile, combined with a tailored allocation of dual-mode CIM resources, CMSwitch allows some arrays to operate in memory mode. In contrast, prior compilation frameworks have not adequately considered this issue, leading to under utilization of hardware resources. Although acceleration benefits for certain CNNs may be less pronounced, the introduction of the resource-switching capabilities enables better handling of the diverse computational and memory demands across different CNN layers. CMSwitch’s dual-mode-aware compilation optimization ensures more efficient handling of resource demands.

The results demonstrate CMSwitch’s versatility across a variety of neural networks with differing hardware resource requirements. It effectively adjusts CIM array modes based on workload characteristics, consistently showing advantages across various batch sizes. This adaptability reflects CMSwitch’s capability to optimally allocate compute and memory resources, ensuring performance acceleration at different scales and workloads.

5.3 Dual-mode Switch Result Demonstration

This section presents the allocation of compute/memory arrays after compilation. As shown in Figure 15, the dashed boxes show the network segment, and the pie charts show the proportion of compute/memory arrays allocated.

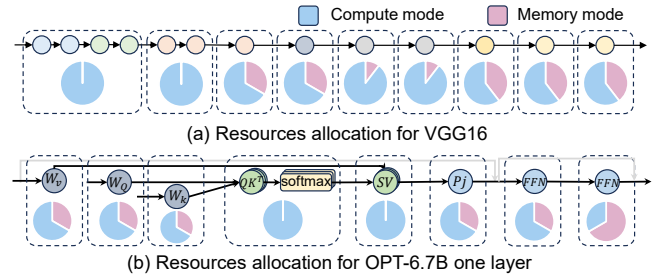


Figure 15. Resources allocation for applications.

For the convolution-based VGG16 (Figure 15(a)), the segmentation results divided the topologically sorted convolution operators 1-4 and 5-6 into two segments, while each of the remaining convolution operators was placed in its own segment. This segmentation aligns with intuitive expectations. Due to the increasing number of feature map channels in VGG, the earlier layers require fewer compute arrays for weight mapping compared to the later layers, making it feasible to group multiple operators into one segment for pipeline parallelism. Meanwhile, our MIP-based compute/memory allocation strategy provides customized hardware resource support for different network segments. As shown in the Figure 15(a), the compiled results allocate more compute arrays for the earlier convolutional operators, facilitating parallel computation of multiple operators. With fewer channels, the amount of data required to be loaded is relatively small, and the original on-chip buffer can support the data transfer needs, thus more arrays are used in compute mode. For the later network layers, especially the final convolutional layers, with more input channels, more data is needed in a single MAC computation. Therefore, our model tends to set some CIM arrays to memory mode, providing greater bandwidth for data retrieval.

For the transformer-based model OPT-6.7B, the allocation of compute/memory arrays for one layer is shown in the Figure 15 (b). In the standard matrix multiplication, like QKV

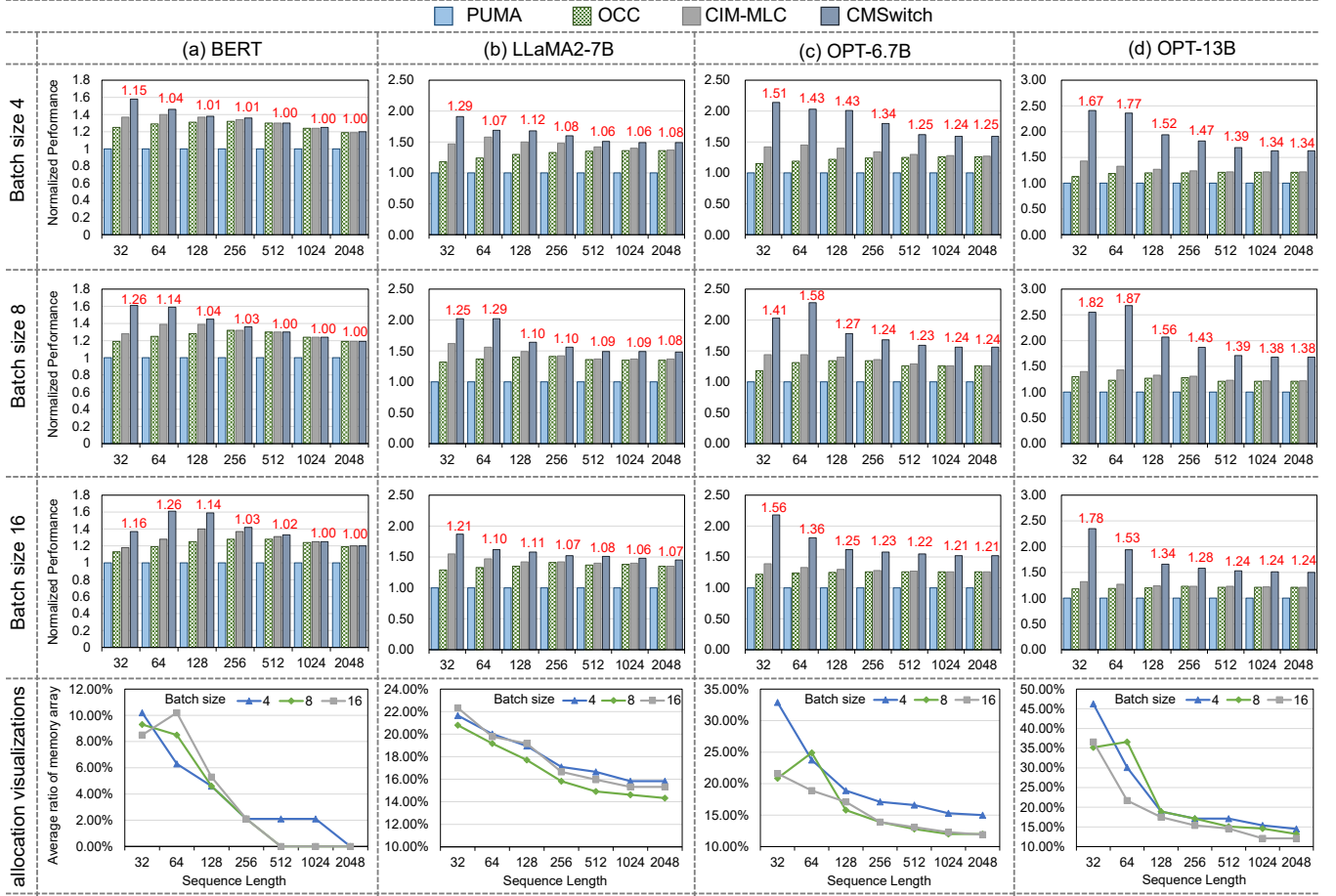


Figure 16. Effectiveness for various workload scales. The horizontal header indicates the model and the vertical header indicates the batch size. (top three rows) Visualization of the memory/compute allocation sensitivity across different sequence lengths for the transformer-based model. (last row) The number in red indicates the speedup compared to the CIM-MLC.

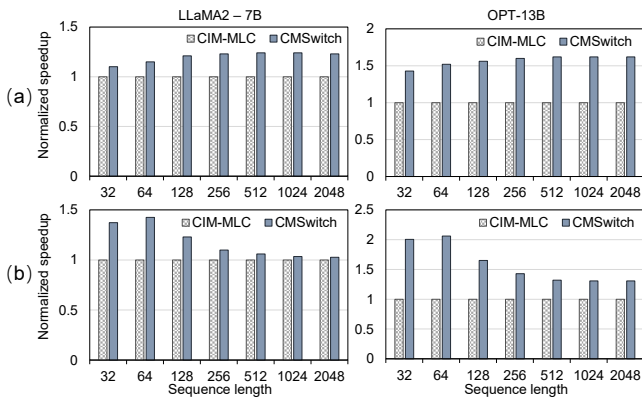


Figure 17. Effectiveness for generative models. (a) Fixed input sequence length; (b) Fixed output sequence length.

generation and feed-forward net (FFN), CMSwitch allocates 33%~67% CIM arrays in the memory mode based on the operators' demands. For example, the last FFN operator needs

more input data for each computation and is therefore allocated slightly more memory arrays. In contrast, for attention calculations, which consume data immediately after computation, more compute arrays are allocated. Moreover, after calculating the K value, some K data is stored on-chip as it owns some CIM array in memory mode. Once the respective CIM arrays switch from memory to compute mode, QK^T computations can proceed directly in place, aligning to minimize data transfer. This strategic allocation demonstrates CMSwitch's ability to optimize resource usage and improve performance effectively.

5.4 Effectiveness for Various Workload Scale

In this section, we analyze the impact of workload scale on the compilation results of CMSwitch. As depicted in Figure 16, we evaluate BERT-large, Llama 2-7B, OPT-6.7B, and OPT-13B with batch sizes ranging from 4 to 16 and input/output sequence lengths from 32 to 2048. Meanwhile, we visualize the memory/compute allocation sensitivity across

different sequence lengths in Figure 16 last row. The average proportion of arrays operating in memory mode across all segments serves as an indicator of the overall resource allocation strategy, though variations may occur within individual segments.

As shown in Figure 16, across various batch sizes, CMSwitch achieves an average performance improvement of $1.19\times$ to $1.03\times$ over CIM-MLC for BERT when sequence lengths range from 32 to 256. For sequence lengths exceeding 512, CMSwitch demonstrates equivalent performance to CIM-MLC, a trend that remains consistent across different batch sizes. Furthermore, the last row of Figure 16 reveals that, as the sequence length increases, the average ratio of arrays in memory mode gradually decreases to zero. This trend aligns with the characteristics of BERT, where the arithmetic intensity increases with longer sequence lengths, necessitating a shift toward compute resources for the corresponding computations. Therefore, as the sequence length extends, CMSwitch’s performance converges with that of CIM-MLC, as we adopt its kernel optimizations.

For generative models, across various batch sizes, CMSwitch yields average performance gains of $1.25\times$ to $1.08\times$ for LLaMA2-7B, $1.50\times$ to $1.23\times$ for OPT-6.7B, and $1.76\times$ to $1.32\times$ for OPT-13B. However, as sequence lengths extend to 512, the speedup diminishes, with fewer arrays operating in memory mode, after which performance stabilizes. For instance, when evaluating OPT-6.7B with a batch size of 8, and varying input and output sequence lengths from 32 to 2048, the speedups compared to CIM-MLC are $1.41\times$, $1.58\times$, $1.27\times$, $1.23\times$, $1.22\times$, and $1.22\times$, while the corresponding average memory mode array ratios decrease from 20.8% to 12.0%. As the input and output sequence lengths increase, the arithmetic intensity during the input processing grows, prompting more arrays to switch to compute mode. This shift reduces the dual-mode switching benefits, resulting in lower overall speedups compared to CIM-MLC, which statically configures all arrays in compute mode. To further assess the performance of CMSwitch for generative models during different inference stages, we use LLaMA2-7B and OPT-13B as benchmarks. We fix the input length at 128 and vary the output length from 32 to 2048, and vice versa, to observe the speedup. As shown in Figure 17, when the input length is fixed, CMSwitch achieves a performance improvement of $1.10\times$ to $1.24\times$ over CIM-MLC for LLaMA-2 7B and $1.43\times$ to $1.62\times$ for OPT-13B, with a nearly consistent speedup as the output sequence length increases. This consistency arises because the decode phase that generates the output sequence processes tokens incrementally, leaving the arithmetic intensity unaffected by changes in output length. Additionally, the varying memory space required by caching KV with longer output sequences benefits from dynamic mode switching. Conversely, when the output length is fixed, CMSwitch’s performance diminishes as the input sequence length increases

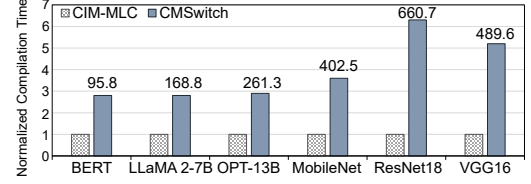


Figure 18. Compilation overhead of different workloads. The number over the bar is the absolute number of compilation time (in s).

for both models, driven by the higher arithmetic intensity, which demands additional compute resources.

Evaluations with varying sequence lengths demonstrate that CMSwitch exhibits adaptability to diverse workload demands by allocating memory and compute resources efficiently, yielding customized compilation results. Moreover, CMSwitch better supports applications with dynamically changing on-chip memory requirements, as existing compilation works typically treat all arrays in compute mode, overlooking their potential use as scratchpads memory.

5.5 Cost and Scalability Analysis

Dual-mode switch overhead. In our evaluation, the dual-mode switch process introduces negligible overhead, contributing around 3% - 5% to the total execution time when providing considerable performance improvement. The dual-mode switch process takes time to configure the input drivers for the mode switch. This minimal overhead is attributable to the efficient design of the CIM chip, which ensures that the switch between compute and memory modes is both swift and seamless. Furthermore, the performance gains realized through our switching overhead-aware network segment strategy, which optimizes network segment scheduling, outweigh the minor switching costs. This evidence confirms that the dual-mode switch mechanism is a valuable consideration in the CIM compilation optimization process.

Scalability. After adjusting the hardware settings to the PRIME [9] architecture, we evaluated the performance of transformer-based networks. Compared to Dynaplasia, PRIME offers larger and more CIM arrays that can contain large network segments. However, PRIME has higher write overhead as it uses the ReRAM as the memory device. According to our assessment, compared to the CIM-MLC, we achieved speedups of $1.48\times$ for Bert, $1.09\times$ for Llama-7B, and $1.10\times$ for OPT-13B. These results indicate that CMSwitch can provide performance improvements across different target hardware configurations. This adaptability demonstrates the robustness and efficiency of CMSwitch in leveraging the capabilities of various CIM accelerator architectures.

5.6 Compilation overhead

We compare the compilation time of CMSwitch with CIM-MLC to demonstrate CMSwitch's overhead. To reduce variability, each benchmark used in the end-to-end evaluation was compiled 20 times. As illustrated in Figure 18, CMSwitch's compilation time is approximately $2.8\times$ to $6.3\times$ longer than that of CIM-MLC, indicating a higher compilation overhead. This increase stems from an exponentially expanded compilation space, which introduces more opportunities for optimization. Given that the compilation process is a one-time operation, the extended duration is justified by the potential for substantial performance gains through a more thorough exploration of the optimization space. Regarding the sensitivity to compilation overhead, we observe that CNNs like ResNet18 and VGG16 require roughly $2.5\times$ more compilation time compared to transformer-based models. This is attributed to the larger compilation space for CNNs, which feature approximately three times as many convolution types with diverse kernel sizes, whereas transformer-based models allow the compilation results of a single block to be reused across all layers. Consequently, the compilation space for transformers is relatively compact. Therefore, for the evaluated benchmarks, the compilation overhead scales almost linearly with the workload size, even as the optimization space expands. This linear scaling is achieved by leveraging techniques such as impossible-case pruning and non-enumerative solving methods by DP and MIP, which accelerate the exploration of the optimization space and effectively reduce potential overhead.

6 Discussion

Opportunity for general-purpose system. While this work focuses on a standalone CIM accelerator system, exploring the compute/memory dual-functionality of CIM architecture offers significant potential for improving general-purpose system performance. Whether CIM is used as a co-processor or a standalone accelerator, dedicating all of its resources solely to computation may not achieve the optimal operating point for many tasks, due to their varying resource requirements. In a real system enhanced with CIM, tasks that go beyond DNNs will be encountered more frequently [17, 28, 30]. These tasks often involve complex and dynamic memory and computation needs. To address this, it is crucial to adapt the CIM architecture to meet the specific demands of each task. One effective approach lies in dynamically adjusting the allocation of computation capacity, memory size, and bandwidth through mode switching. This flexibility allows the system to better match the resource requirements of diverse workloads. The trend toward tighter integration of memory and computation is key to supporting the growing diversity of memory-intensive tasks. Therefore, leveraging the dual-mode capability of CIM in a more flexible manner is crucial to optimizing system efficiency.

7 Conclusion

In this paper, we presented a novel compilation optimization process tailored for dual-mode CIM chips. By incorporating the compute-memory dual-mode switch into the compilation optimization space, we developed a comprehensive approach that optimizes the memory/compute CIM array allocation for various networks. Our dynamic programming-based network segmentation, coupled with mixed integer programming for operator mapping and scheduling, ensures efficient utilization of CIM resources. Experimental results demonstrate $1.31\times$ performance improvements on average across different models compared to the state-of-the-art CIM compilation work, highlighting the effectiveness of our approach in diverse workloads. By utilizing the dual-mode feature of the CIM processor, our method CMSwitch achieves substantial speedup while maintaining low overhead. This work paves the way for more efficient and flexible use of CIM chips in real-world DNN applications, enhancing the performance and scalability of CIM-based systems.

Acknowledgments

We sincerely thank the anonymous reviewers for their insightful suggestions for improving this paper. This work was partially supported by the National Key R&D Program of China (Grant No. 2023YFB4404400) and the National Natural Science Foundation of China (Grant No. 62222411, 62025404, 62204164). Ying Wang (wangying2009@ict.ac.cn) is the corresponding author.

References

- [1] Junwhan Ahn, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. Pim-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture. *ACM SIGARCH Computer Architecture News*, 43(3S):336–348, 2015.
- [2] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Sapan Agarwal, Matthew Marinella, Martin Foltin, John Paul Strachan, Dejan Milojicic, Wen-Mei Hwu, and Kaushik Roy. Panther: A programmable architecture for neural network training harnessing energy-efficient reram. *IEEE Transactions on Computers*, 69(8):1128–1142, 2020.
- [3] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R Stanley Williams, Paolo Faraboschi, Wen-mei W Hwu, John Paul Strachan, Kaushik Roy, et al. Puma: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, pages 715–731, 2019.
- [4] Junjie Bai, Fang Lu, Ke Zhang, et al. Onnx: Open neural network exchange. <https://github.com/onnx/onnx>, 2019.
- [5] Avishek Biswas and Anantha P Chandrakasan. Conv-sram: An energy-efficient sram with in-memory dot-product computation for low-power convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 54(1):217–230, 2018.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Aspell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [7] Pai-Yu Chen, Xiaochen Peng, and Shimeng Yu. Neurosim: A circuit-level macro model for benchmarking neuro-inspired architectures

- in online learning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(12):3067–3080, 2018.
- [8] Yiran Chen, Yuan Xie, Linghao Song, Fan Chen, and Tianqi Tang. A survey of accelerator architectures for deep neural networks. *Engineering*, 6(3):264–274, 2020.
 - [9] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory. In *Proceedings of the 43rd International Symposium on Computer Architecture, ISCA '16*, pages 27–39, Piscataway, NJ, USA, 2016. IEEE Press.
 - [10] Yu-Der Chih, Po-Hao Lee, Hidehiro Fujiwara, Yi-Chun Shih, Chia-Fu Lee, Rawan Naous, Yu-Lin Chen, Chieh-Pu Lo, Cheng-Han Lu, Haruki Mori, et al. 16.4 an 89tops/w and 16.3 tops/mm² all-digital sram-based full-precision compute-in memory macro in 22nm for machine-learning edge applications. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 64, pages 252–254. IEEE, 2021.
 - [11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
 - [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
 - [13] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
 - [14] Andi Drebes, Lorenzo Chelini, Oleksandr Zinenko, Albert Cohen, Henk Corporaal, Tobias Grosser, Kanishkan Vadivel, and Nicolas Vasilache. Tc-cim: Empowering tensor comprehensions for computing-in-memory. In *IMPACT 2020-10th International Workshop on Polyhedral Compilation Techniques*, 2020.
 - [15] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramaniyan, Ravi Iyer, Dennis Sylvester, David Blaauw, and Reetuparna Das. Neural cache: Bit-serial in-cache acceleration of deep neural networks. In *2018 ACM/IEEE 45th annual international symposium on computer architecture (ISCA)*, pages 383–396. IEEE, 2018.
 - [16] Hamid Farzaneh, João Paulo Cardoso de Lima, Mengyuan Li, Asif Ali Khan, Xiaobo Sharon Hu, and Jeronimo Castrillon. C4cam: A compiler for cam-based in-memory accelerators. *arXiv preprint arXiv:2309.06418*, 2023.
 - [17] Daichi Fujiki, Scott Mahlke, and Reetuparna Das. Duality cache for data parallel acceleration. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 397–410, 2019.
 - [18] Hidehiro Fujiwara, Haruki Mori, Wei-Chang Zhao, Mei-Chen Chuang, Rawan Naous, Chao-Kai Chuang, Takeshi Hashizume, Dar Sun, Chia-Fu Lee, Kerem Akarvardar, et al. A 5-nm 254-tops/w 221-tops/mm² fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous mac and write operations. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 1–3. IEEE, 2022.
 - [19] An Guo, Xin Si, Xi Chen, Fangyuan Dong, Xingyu Pu, Dongqi Li, Yongliang Zhou, Lizheng Ren, Yeyang Xue, Xueshan Dong, et al. A 28nm 64-kb 31.6-tflops/w digital-domain floating-point-computing-unit and double-bit 6t-sram computing-in-memory macro for floating-point cnns. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 128–130. IEEE, 2023.
 - [20] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*, 2021.
 - [21] Jianhui Han, Xiang Fei, Zhaolin Li, and Youhui Zhang. Polyhedral-based compilation framework for in-memory neural network accelerators. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 18(1):1–23, 2021.
 - [22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
 - [23] Biresh Kumar Joardar, Bing Li, Janardhan Rao Doppa, Hai Li, Partha Pratim Pande, and Krishnendu Chakrabarty. Regent: A heterogeneous reram/gpu-based architecture enabled by noc for training cnns. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 522–527. IEEE, 2019.
 - [24] Sangjin Kim, Zhiyong Li, Soyeon Um, Wooyoung Jo, Sangwoo Ha, Juhyoung Lee, Sangyeob Kim, Donghyeon Han, and Hoi-Jun Yoo. 16.5 dynaplasia: An edram in-memory-computing-based reconfigurable spatial accelerator with triple-mode cell for dynamic resource switching. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 256–258. IEEE, 2023.
 - [25] Manuel Le Gallo, Riduan Khaddam-Aljameh, Milos Stanisavljevic, Athanasios Vasilopoulos, Benedikt Kersting, Martino Dazzi, Geethan Karunaratne, Matthias Brändli, Abhairaj Singh, Silvia M Mueller, et al. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nature Electronics*, 6(9):680–693, 2023.
 - [26] Bing Li, Linghao Song, Fan Chen, Xuehai Qian, Yiran Chen, and Hai Helen Li. Reram-based accelerator for deep learning. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 815–820. IEEE, 2018.
 - [27] Shuangchen Li, Dimin Niu, Krishna T Malladi, Hongzhong Zheng, Bob Brennan, and Yuan Xie. Drisa: A dram-based reconfigurable in-situ accelerator. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 288–301, 2017.
 - [28] Elliot Lockerman, Axel Feldmann, Mohammad Bakhshalipour, Alexandru Stanescu, Shashwat Gupta, Daniel Sanchez, and Nathan Beckmann. Livia: Data-centric computing throughout the memory hierarchy. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 417–433, 2020.
 - [29] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. 2020.
 - [30] Marcelo Orenes-Vera, Esin Tureci, David Wentzlaff, and Margaret Martonosi. Dalorex: A data-local program execution and architecture for memory-bound applications. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 718–730. IEEE, 2023.
 - [31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
 - [32] Songyun Qu, Bing Li, Ying Wang, Dawen Xu, Xiangdong Zhao, and Lei Zhang. Raqu: An automatic high-utilization cnn quantization and mapping framework for general-purpose rram accelerator. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.
 - [33] Songyun Qu, Shixin Zhao, Bing Li, Yintao He, Xuyi Cai, Lei Zhang, and Ying Wang. Cim-mlc: A multi-level compilation stack for computing-in-memory accelerators. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 185–200, 2024.
 - [34] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.

- [35] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
- [36] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.
- [37] Vivek Seshadri, Donghyuk Lee, Thomas Mullins, Hasan Hassan, Amirali Boroumand, Jeremie Kim, Michael A Kozuch, Onur Mutlu, Phillip B Gibbons, and Todd C Mowry. Ambit: In-memory accelerator for bulk bitwise operations using commodity dram technology. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 273–287, 2017.
- [38] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R Stanley Williams, and Vivek Srikumar. Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. *ACM SIGARCH Computer Architecture News*, 44(3):14–26, 2016.
- [39] Adam Siemieniuk, Lorenzo Chelini, Asif Ali Khan, Jeronimo Castrillon, Andi Drebes, Henk Corporaal, Tobias Grosser, and Martin Kong. Occ: An automated end-to-end machine learning optimizing compiler for computing-in-memory. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(6):1674–1686, 2021.
- [40] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [41] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. Pipelayer: A pipelined rram-based accelerator for deep learning. In *2017 IEEE international symposium on high performance computer architecture (HPCA)*, pages 541–552. IEEE, 2017.
- [42] Samuel D Spetalnick, Muya Chang, Brian Crafton, Win-San Khwa, Yu-Der Chih, Meng-Fan Chang, and Arijit Raychowdhury. A 40nm 64kb 26.56 tops/w 2.37 mb/mm² rram binary/compute-in-memory macro with 4.23 x improvement in density and > 75% use of sensing dynamic range. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 1–3. IEEE, 2022.
- [43] Shrihari Sridharan, Jacob R Stevens, Kaushik Roy, and Anand Raghunathan. X-former: In-memory acceleration of transformers. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.
- [44] Xiaotian Sun, Xinyu Wang, Wanqian Li, Lei Wang, Yinhe Han, and Xiaoming Chen. Pimcomp: A universal compilation framework for crossbar-based pim dnn accelerators. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023.
- [45] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [47] Weier Wan, Rajkumar Kubendran, Clemens Schaefer, Sukru Burc Eryilmaz, Wenqiang Zhang, Dabin Wu, Stephen Deiss, Priyanka Raina, He Qian, Bin Gao, et al. A compute-in-memory chip based on resistive random-access memory. *Nature*, 608(7923):504–512, 2022.
- [48] Dewei Wang, Chuan-Tung Lin, Gregory K Chen, Phil Knag, Ram K Krishnamurthy, and Mingoo Seok. Dimc: 2219tops/w 2569f2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware. In *2022 IEEE international solid-state circuits conference (ISSCC)*, volume 65, pages 266–268. IEEE, 2022.
- [49] Wm A Wulf and Sally A McKee. Hitting the memory wall: Implications of the obvious. *ACM SIGARCH computer architecture news*, 23(1):20–24, 1995.
- [50] Lixue Xia, Boxun Li, Tianqi Tang, Peng Gu, Pai-Yu Chen, Shimeng Yu, Yu Cao, Yu Wang, Yuan Xie, and Huazhong Yang. Mnsim: Simulation platform for memristor-based neuromorphic computing system. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(5):1009–1022, 2017.
- [51] Bonan Yan, Jeng-Long Hsu, Pang-Cheng Yu, Chia-Chi Lee, Yaojun Zhang, Wenshuo Yue, Guoqiang Mei, Yuchao Yang, Yue Yang, Hai Li, et al. A 1.041-mb/mm² 27.38-tops/w signed-int8 dynamic-logic-based adc-less sram compute-in-memory macro in 28nm with reconfigurable bitwise operation for ai and embedded applications. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 188–190. IEEE, 2022.
- [52] Xiaoxuan Yang, Bonan Yan, Hai Li, and Yiran Chen. Retransformer: Reram-based processing-in-memory architecture for transformer acceleration. In *Proceedings of the 39th International Conference on Computer-Aided Design*, pages 1–9, 2020.
- [53] Jinshan Yue, Xiaoyu Feng, Yifan He, Yuxuan Huang, Yipeng Wang, Zhe Yuan, Mingtao Zhan, Jiaxin Liu, Jian-Wei Su, Yen-Lin Chung, et al. 15.2 a 2.75-to-75.9 tops/w computing-in-memory nn processor supporting set-associate block-wise zero skipping and ping-pong cim with simultaneous computation and weight updating. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 64, pages 238–240. IEEE, 2021.
- [54] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.